and test code whenever changes are pushed to the repository.

- *Automated builds:* Define workflows that compile and build the application automatically on each code commit. This ensures that the codebase is always in a buildable state.
- *Automated testing:* Automated tests are set up to run as part of the CI pipeline. This helps in identifying and addressing issues early in the development cycle, reducing the chances of defects making it to production.

Continuous deployment (CD): Continuous deployment (CD) extends CI by automating the deployment of applications to production or staging environments. GitHub Actions enable teams to create CD pipelines that deploy applications automatically based on predefined criteria.

- *Automated deployments:* Create workflows that deploy the application to various environments (e.g., development, staging, production) whenever changes are merged into specific branches.
- *Approval gates:* Implement approval steps in the deployment pipeline to ensure that critical deployments require manual approval, adding an extra layer of control and security.

Code quality and testing: Maintaining high code quality is essential for delivering reliable software. GitHub Actions can be configured to run various code quality checks and testing frameworks to ensure adherence to coding standards and best practices.

- *Static code analysis:* Integrate tools like ESLint, Pylint, or SonarQube to perform static code analysis, identifying potential issues such as code smells, security vulnerabilities, and maintainability concerns.
- *Unit and integration testing:* Define workflows to run unit tests and integration tests, ensuring that individual components and their interactions function as expected.

Collaboration and visibility: By using GitHub Actions, teams can improve collaboration and visibility into the development process. Workflows defined in the repository are accessible to all team members, promoting transparency and collective ownership.

- *Pull request workflows:* Set up workflows that run tests and checks on pull requests, providing immediate feedback to contributors about the impact of their changes.
- *Notifications and alerts:* Configure workflows to send notifications to relevant stakeholders (e.g., via email, Slack) about the status of builds, tests, and deployments, keeping everyone informed.

Cost efficiency and resource optimisation: Automating repetitive and manual tasks with GitHub Actions can lead to significant cost savings and better resource utilisation. Teams can focus more on innovation and less on operational overhead.

- *Infrastructure as Code (IaC):* Use GitHub Actions to automate the provisioning and management of infrastructure using IaC tools like Terraform or Ansible, ensuring consistent and repeatable infrastructure setups.
- *Resource management:* Automate the scaling of resources based on demand, optimising costs by spinning up resources only when needed and tearing them down when not in use.

Getting started with GitHub Actions

Creating a workflow: To create a workflow, you need to define a YAML file within the *.github/workflows* directory of your repository. This file describes the sequence of steps to be executed in response to specific events.

```
name: CI Pipeline

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v2

      - name: Set up Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '14'

      - name: Install Dependencies
        run: npm install

      - name: Run Tests
        run: npm test
```

Choosing triggers: Specify the events that should trigger the workflow. Common triggers include *push*, *pull_request*, and *schedule*.

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  schedule:
    - cron: '0 0 * * *' # Runs daily at midnight
```

Continued to page...51